Centro Paula Souza
Etec Vasco Antonio Venchiarutti – Jundiaí - SP
Técnico em Desenvolvimento de Sistemas Integrado ao Ensino Médio – Out/2026

Henrique Silvestre Martin
Isabella Fernanda da Silva Barbosa

Desenvolvimento de Sistemas: Componente *"DataGridView"* no *Visual Studio C# desktop*

Professoras Cristiane Fernandes e Adani Cusin Sacilotti

Jundiaí
2026

INTRODUÇÃO

O presente trabalho tem como objetivo apresentar, de forma prática e organizada, uma pesquisa sobre as aplicações que apresentam registros em forma de tabela e que permitem ao usuário consultar e alterar dados com rapidez. O desenvolvimento de aplicações *desktop* exige interfaces que permitam apresentar, organizar e manipular informações de maneira eficiente, sendo que nesse contexto, os controles de grade do .NET — chamados *grid* — organizam informações em linhas e colunas e podem integrar apresentação, edição, seleção e validação. Tendo como objetivo apresentar os conceitos fundamentais do controle *DataGridView* do *Windows Forms*, o material aborda suas formas de vinculação de dados, operações essenciais, possibilidades de formatação visual e boas práticas de desempenho e organização do código.

1. O QUE É O DATAGRID

O *DataGridView* (presente no *namespace System.Windows.Forms.*) é um componente visual do *framework* .NET (utilizado principalmente em aplicações *desktop Windows Forms*) projetado para exibir e editar dados. Sua principal função é apresentar dados em uma tabela, na qual cada linha representa um registro ou item e cada coluna representa um campo, atributo ou operação, permitindo a visualização organizada dos registros, além da ordenação, seleção e personalização de células e colunas. No ecossistema .NET, é importante diferenciá-lo do *DataGrid* do *Windows Forms*, já que eles têm suas APIs e formas de definir interfaces diferentes. O DataGridView "substitui" a versão antiga com amplas melhorias arquiteturais, oferecendo suporte a modelos de células flexíveis, formatação refinada e desempenho otimizado. Além disso, ao comparar o *DataGridView* com o componente *DataGrid* do WPF (*Windows Presentation Foundation*), nota-se que o WPF tende a ser baseado em *templates* de estilos e XAML (*DataTemplates*), enquanto o *Windows Forms* foca em programação imperativa orientada a eventos e propriedades diretas de controle.

2. CONCEITOS BÁSICOS

O funcionamento do *DataGridView* baseia-se fortemente no conceito de *Data Binding* (Vinculação de Dados), permitindo sincronizar o componente com estruturas de dados sem a necessidade de preenchimento manual linha por linha. Dessa maneira, ele conecta a fonte de dados ao controle. Em Windows Forms, a forma básica é, por exemplo, `dataGridView1.DataSource = lista;` ou com o uso de um *BindingSource* como intermediário

para navegação e gestão da fonte funcionando com o código: BindingSource bindingSource = new BindingSource(); bindingSource.DataSource = minhaLista; dataGridView1.DataSource = bindingSource;. As principais fontes de dados são: *List<T> / BindingList<T>* (ideais para coleções de objetos orientados a objetos), *DataTable / DataSet* (estruturas relacionais em memória originadas do ADO.NET.) e os bancos de dados relacionais (conexões diretas via *Entity Framework* ou comandos SQL parametrizados). Em relação à estrutura do *DataGridView*, é possível destacar que cada linha representa um registro ou item da coleção e cada coluna exibe uma propriedade ou campo que podem ser geradas automaticamente a partir dos campos disponíveis ou definidas anteriormente, permitindo definir seus títulos, tipos e propriedades.

3. OPERAÇÕES ESSENCIAIS

O *DataGridView* permite realizar operações de inclusão, edição, exclusão e seleção de registros com a integração de eventos da interface com fonte de dados subjacente (como uma *List<T>*, *DataTable*, ou outras mencionadas anteriormente). A inclusão pode ocorrer diretamente no componente, quando essa funcionalidade está habilitada, ou por meio de formulários e botões externos, a edição permite modificar os valores das células e a exclusão pode ser realizada a partir da linha selecionada. Entretanto, alterar um valor exibido no *grid* não significa necessariamente que a informação foi salva no banco de dados, é necessário atualizar a fonte de dados. Quando associado a uma fonte que suporta notificações de alteração (como *BindingList*), é necessário usar as propriedades como AllowUserToAddRows (adicionar itens) e AllowUserToDeleteRows (deletar itens), além de ReadOnly (que impede que o usuário altere ou apague algo pelo teclado), os quais controlam permissões de interação quando a fonte é atualizada.

Para capturar quando o usuário clica ou seleciona qualquer célula, linha ou coluna, é utilizado o evento CellClick, que fornece o índice da linha e da coluna clicada. Já para quando o usuário altera o valor de uma célula e confirma a edição, é utilizado o CellValueChanged, que rastreia as alterações de dados e sinaliza que os valores das células foram alterados na interface/*grid.*

4. FORMATAÇÃO VISUAL BÁSICA

A formatação visual contribui para a organização das informações e para a facilitação da leitura dos dados, portanto, é importante utilizar alguns recursos estéticos para otimizar a compreensão do usuário. Dentre os recursos, os principais são:

- **Configuração das cores de linhas e células:** propriedades como DefaultCellStyle.BackColor e ForeColor, utilizadas da seguinte maneira: dataGridView1.Columns/Rows[0].DefaultCellStyle.BackColor=Color.x; DataGridView1.Columns/Rows[0].DefaultCellStyle.ForeColor=Color.DarkBlue; (intercalando as cores das colunas com "*Columns*" ou linhas com "*Rows*").
- **Estilo alternância de cores (zebra):** pode-se obter esse estilo ativando a propriedade AlternatingRowsDefaultCellStyle ou configurando pelo seguinte código: dataGridView1.DefaultCellStyle.BackColor=Color.White; dataGridView1.AlternatingRowsDefaultCellStyle.BackColor = Color.LightGray;.
- **Configuração dos cabeçalhos:** desativando o estilo padrão anteriormente, o código é dataGridView1.ColumnHeadersDefaultCellStyle.BackColor=Color.Navy; dataGridView1.ColumnHeadersDefaultCellStyle.ForeColor = Color.White;.
- **Largura das colunas e linhas:** pode ser definido nas propriedades do DataGridView ou pelo código dataGridView1.Columns[0 ou nomedacolunaespecífica].Width = 100; e dataGridView1.RowTemplate.Height = 30; (alterar todas as linhas).
- **Alinhamento:** para alinhar os conteúdos por coluna é necessário usar o DataGridViewContentAlignment, cujas principais opções de alinhamento são *MiddleLeft, TopLeft, BottomLeft, MiddleCenter, TopCenter, BottomCenter, MiddleRight, TopRight, BottomRight*.

5. FORMATAÇÃO VISUAL AVANÇADA

Dentre as outras funções importantes, destacam-se a formatação condicional, onde para alterar cores de fundo com base nos valores (por exemplo, destacar em vermelho salários abaixo do mínimo), utilizamos o evento CellFormatting. Além disso, para personalizar colunas usa-se: DataGridViewComboBoxColumn para exibir listas suspensas dentro da célula para seleção rápida de opções; DataGridViewCheckBoxColumn para inserir caixas de seleção para campos booleanos e DataGridViewButtonColumn com botões interativos por linha para execuções específicas (como exclusão rápida ou visualização de detalhes). Para os estilos por célula/linha: CellStyle é utilizado para alterar cores de fundo, fontes e alinhamento de células com base em regras simples e o RowPrePaint é o evento mais para indicado desenhar gradientes, bordas customizadas, formas geométricas ou texturas na linha. Além da formatação de valores numéricos, que define o formato das células com a propriedade DefaultCellStyle.Format, permitindo formatos como o de moeda C ou C2, N2 (duas casas decimais), P (porcentagem),

para datas dd/MM/yyyy (data, mês e ano), dd/MM/yyyy HH:mm (data com horas e minutos). Todas essas funções contribuem na personalização da aparência geral do *DataGridView*.

6. BOAS PRÁTICAS E DESEMPENHO

O trabalho com esse componente exige certa cautela para evitar travamentos na interface, sendo necessário alguns cuidados como evitar AutoSizeColumnsMode, desativar a criação automática de colunas e limitar resultados com paginação. Além disso, recomenda-se o VirtualMode = true, que gerencia o cache de dados manualmente reduzindo o consumo de memória e preferir o compartilhamento de estilos aos objetos de estilo para cada célula ou linha, sendo essas práticas que ajudam a manter a formatação organizada e visualmente agradável, mesmo ao trabalhar com grandes quantidades de dados.

CONCLUSÃO

Com base no exposto, é possível concluir que o *DataGridView*, quando bem formatado, é uma excelente ferramenta de visualização de dados, já que o usuário consegue localizar informações importantes e editar um banco de dados com facilidade. Dessa maneira, a interface é determinante para a experiência eficiente do usuário, integrando as funcionalidades às técnicas de formatação. Por isso, é indispensável considerar que clareza visual e organização contribuem na legibilidade das informações e, consequentemente, devem ser planejadas pelo desenvolvedor para garantir a usabilidade e acessibilidade visual.

REFERÊNCIAS

MICROSOFT. **DataGridView Classe.** Disponível em: <https://learn.microsoft.com/pt-br/dotnet/api/system.windows.forms.datagridview?view=windowsdesktop-10.0>. Acesso em: 15 out 2026.

MICROSOFT. **Formatação básica e estilo no controle DataGridView - Windows Forms**. Disponível em: <https://learn.microsoft.com/pt-br/dotnet/desktop/winforms/controls/basic-formatting-and-styling-in-the-windows-forms-datagridview-control>. Acesso em: 16 out 2026.

MICROSOFT. **Diferenças entre DataGridView e controles DataGrid - Windows Forms.** Disponível em: <https://learn.microsoft.com/pt-br/dotnet/desktop/winforms/controls/differences-between-the-windows-forms-datagridview-and-datagrid-controls> Acesso em: 16 out 2026.

MUNHOZ, E. **C# DataGridView: Trabalhando com DataGridView no Microsoft Visual Studio**. Disponível em: <https://www.devmedia.com.br/csharp-datagridview-trabalhando-com-datagridview-no-microsoft-visual-studio/31137>. Acesso em: 16 out 2026.